D6 Verifying AI output

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 2
D6 Verifying AI output 3
D6-L1 Read before you run, and check each API 3
D6-L2 Small data with a known answer, failing test first 7
D6-L3 Ask for assumptions, then compare a second approach 11
Track D best-practice checklist 16
Appendix: solutions and answer keys 17

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D6 Verifying AI output

Roadmap: Every month. A workflow to check AI code before you trust it.

D6-L1 Read before you run, and check each API

Concept

Track D so far showed what goes wrong in AI-written code. Module D6 is the routine that catches it. Step one happens before the code runs at all.

Why read first. Running code is the fastest check for invented names, but running is not free. A suggested cell might overwrite a table, drop a column or read a whole year of data. Reading takes two minutes. Restoring a bronze table takes a morning, if you can do it at all.

The read-through:

1. **Restate the goal** in one sentence before you look at the code. "Append wave W14 deliveries to bronze with audit columns." You now have something to check against.
2. **List every external name**: imports, functions, methods, parameters, tables and columns. An `ast` walk lists the calls for you (exercise 4).
3. **Check each name** against the docs for your version and against the table schema. `DESCRIBE TABLE`, `df.columns`, `hasattr`, `inspect.signature` and `help()` answer most questions in seconds.
4. **Find side effects**: `mode("overwrite")`, `DROP`, `DELETE`, `TRUNCATE`, `VACUUM`, writes to production names. Each needs a reason that matches the goal.
5. **Check the scope**: filters on tracker and wave, no `collect()` on large data, no loop that calls `count()` per item.

Then run it, on development data first, in a dev schema: `research_dev`, not `research_bronze`.

Where assistants go wrong most in this course's work: the write mode, three-level table names, the filter keys, and parameters that exist but mean something else. `mode("overwrite")` on an append-only bronze table replaces every earlier delivery. It runs without error.

A read-through does not prove code correct. It removes the mistakes that are visible on the page, so testing (lesson D6-L2) can focus on the ones that are not.

Why it matters

An assistant rewrote a bronze ingestion cell to "simplify" it. The new cell used `mode("overwrite")`. It ran in production and succeeded, replacing 38 waves of append-only deliveries with one. Delta time travel recovered the table because retention had not yet expired. A two-minute read, looking for side effects, would have found the one word that mattered.

Worked example: weak code

```
1 # Suggested by an assistant: "load the new delivery"
2 delivery = spark.read.format("csv").option("header", True).load(path)
3 delivery.write.mode("overwrite").saveAsTable("research_bronze.nat_raw")
```

1. **Line 1.** No stated goal to check against.
2. **Line 3.** Overwrite on an append-only table replaces every earlier delivery. It runs cleanly.

Worked example: strong code

```
1 # Goal: append the W14 delivery to bronze with audit columns. Bronze is
    append-only.
2 # Checked: format("csv").option("header", True) and saveAsTable (PySpark
    4.0 API reference).
3 from pyspark.sql import functions as F
4
5 delivery = spark.read.format("csv").option("header", True).load(path)
6 delivery = delivery.withColumn("source_file",
    F.col("_metadata.file_path"))
7 delivery.write.mode("append").saveAsTable("research_dev.nat_raw")
```

1. **Line 1.** The goal, written first, makes the write mode checkable.
2. **Line 7.** Append, and into a dev schema until the run has been checked.

Restate the goal, list and check every name, hunt for side effects, then run on development data.

Recap card

- **Five steps of the read-through** Restate the goal, list names, check each name, find side effects, run on dev data.
- **Side effects to hunt for** mode('overwrite'), DROP, DELETE, TRUNCATE, VACUUM, writes to production names.
- **Quick existence checks in a notebook** hasattr for names, inspect.signature(...).parameters for parameters, DESCRIBE TABLE or df.columns for columns.
- **What a read-through does not do** Prove correctness. It clears visible mistakes so tests can find the rest.

Practice

Exercise 1. Multiple choice (level 1)

An assistant gives you a cell that writes to `research_bronze.nat_raw`. What do you check first?

- a) The write mode and the target, against the goal and the table's rules
- b) Whether the code is formatted with ruff
- c) How long it will take
- d) Nothing: run it and read the result

Why?

- r1) Side effects on a production table cannot always be undone. Bronze is append-only.
- r2) Assistants always pick the wrong write mode.

Exercise 2. Put in order (level 2)

Order the read-through of an AI-written cell.

- Restate the goal in one sentence
- Find side effects: write modes, DROP, DELETE
- Check each name against the docs for your version and the schema
- Run it on development data in a dev schema
- List every function, parameter, table and column

Exercise 3. Fill the gap (level 2)

In a notebook, confirm that a suggested function and parameter exist.

```
import inspect
from pyspark.sql import functions as F

_____(1)(F, "try_divide")
"validate" in inspect._____(2)(pd.DataFrame.merge).parameters
```

Exercise 4. Write the function (level 3)

Write `called_names(source: str) -> list[str]` that parses Python source with `ast` and returns the dotted names of every call, such as `spark.read.format` or `F.col`, unique and sorted. Ignore calls whose target is not a name or attribute chain (for example a call on a subscript). This gives you the checklist for step 2 of the read-through.

What you need to know.

`ast.parse(source)` turns Python code into a tree of nodes. `ast.walk(tree)` visits every node. Three node types matter here:

- `ast.Call` is a call. Its `.func` is the thing being called.
- `ast.Name` is a plain name such as `print`. Its `.id` is the text.
- `ast.Attribute` is `something.attr`. Its `.attr` is the last part and its `.value` the node before the dot.

```
import ast

func = ast.parse("os.path.join(a)").body[0].value.func
func.attr, func.value.attr, func.value.value.id   # ('join', 'path', 'os')
```

Test a node's type with `isinstance(node, ast.Call)`.

Your code must pass these checks:

- Return dotted names for name and attribute chains. A call on a call result (.load on format(...)) has no plain dotted name: skip it.
- Each name once, sorted.
- Skip calls on subscripts.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Dotted name for a Name or Attribute chain, else None.
def _dotted(node: ast.expr) -> str | None: ...

# Sorted unique dotted names of every call in the source.
def called_names(source: str) -> list[str]: ...
```

Starter code:

```
def called_names(source):
    return [line for line in source.splitlines() if "(" in line]
```

Exercise 5. Spot the bug (level 3)

Goal: append this wave's delivery to bronze, which is append-only. Tap the line that breaks the goal, then fix it.

```
from pyspark.sql import functions as F

delivery = spark.read.format("csv").option("header", True).load(path)
delivery = delivery.withColumn("ingestion_timestamp",
    F.current_timestamp())
delivery.write.mode("overwrite").saveAsTable("research_bronze.nat_raw")
```

Write down the line number of the bug. Then write the fix.

Sources

- PySpark DataFrameReader.format/option/load, DataFrameWriter.mode and saveAsTable, and the _metadata.file_path column exist as used. (verified: Local runs on PySpark 4.0.4 (see D1 and D5 claims); _metadata.file_path per Databricks Auto Loader docs (A8-L2).)

D6-L2 Small data with a known answer, failing test first

Concept

After the read-through, run the code where you already know the right answer. That means small data you built by hand, and a test written **before** you accept the code.

Build the known-answer data. Five to ten rows, chosen so every rule in the requirement matters:

- a normal case you can compute in your head,
- a duplicate key, a null, a zero total, an empty group,
- a value exactly on a boundary, such as a weight equal to the cap.

Write the expected output by hand, next to the input. If you cannot work out the answer by hand, the requirement is not clear enough yet, and neither the assistant nor you should write code.

Write the test first, and watch it fail. The order matters:

1. Write the test with the hand-worked expected values.
2. Run it against the current code, or a stub. It must **fail**. A test that passes before the feature exists tests nothing.
3. Accept or write the code. Run the test. It passes.
4. Break the code on purpose, say change `>` to `>=`. The test must fail again. If it still passes, it does not cover that rule.

Step 4 is a quick form of mutation testing. It tells you whether your tests would catch the mistakes an assistant makes.

For Spark code, run the same idea on a local SparkSession. `pyspark.testing.assertDataFrameEqual(actual, expected)` compares two DataFrames. In PySpark 4.0 it ignores row order by default (`checkRowOrder=False`) and compares floats with `rtol=1e-05`. `assertSchemaEqual` compares schemas.

Keep the known-answer tests in the repository. They check every later change, including the next suggestion from an assistant.

Why it matters

An assistant wrote a deduplication step for silver and a test for it. The test passed. A reviewer changed the dedup key from `(respondent_id, wave_id)` to `respondent_id` alone, a real bug, and the test still passed. Its fixture had only one wave. Adding a respondent who appears in two waves made the test fail on the broken key, as it should.

Worked example: weak code

```
1 def test_share():
2     result = share(30, 120)
(1)   assert result is not None
(2)   assert 0 <= result <= 1
```

1. **Line 3.** Passes for almost any implementation, including wrong ones.
2. **Line 4.** A range, not an answer. 0.3 would pass as well as 0.25.

Worked example: strong code

```
1  import pytest
2
3  @pytest.mark.parametrize(
4      ("part", "total", "expected"),
5      [
(1)        (30, 120, 0.25),   # normal case, worked by hand
7          (0, 50, 0.0),      # zero share is a real value
(2)        (5, 0, None),      # zero total has no share
9          (50, 50, 1.0),     # boundary
10     ],
11 )
12 def test_share(part: float, total: float, expected: float | None) -> None:
13     assert share(part, total) == expected
```

1. **Line 6.** An exact answer you computed yourself.
2. **Line 8.** The edge case the requirement names.

Hand-built data, hand-worked answers, a test that fails first, and a deliberate break to prove the test bites.

Recap card

- **Known-answer data** 5 to 10 hand-built rows: normal case, duplicate, null, zero total, empty group, boundary.
- **Test-first order** Write the test, watch it fail, accept the code, watch it pass, break the code on purpose, watch it fail.
- **assertDataFrameEqual defaults (PySpark 4.0)** checkRowOrder=False, rtol=1e-05, atol=1e-08.

- **Why break code on purpose** If the test still passes, it misses that rule. A quick form of mutation testing.

Practice

Exercise 1. Multiple choice (level 1)

You write a test for a feature that does not exist yet, and it passes. What does that tell you?

- a) The test does not check the feature
- b) The feature already works
- c) The test is well written
- d) The code is fast

Why?

- r1) A test must fail when the behaviour it checks is missing. Passing without the feature means it asserts something else.
- r2) Tests always fail the first time they run.

Exercise 2. Multiple choice (level 2)

Your tests pass. You change `w > cap` to `w >= cap` in the code on purpose, and the tests still pass. What do you add?

- a) A case with a weight exactly equal to the cap
- b) More cases with weights far above the cap
- c) A timeout on the test
- d) Nothing: the tests pass

Why?

- r1) > and >= differ only on the boundary, so only a boundary case can tell them apart.
- r2) Mutations always need more data.

Exercise 3. Fill the gap (level 2)

Compare a Spark result with the expected rows in a local test, ignoring row order.

```
from pyspark.testing import _____(1)

expected = spark.createDataFrame([("W01", 0.25)], "wave_id string, share
    double")
assertDataFrameEqual(result, expected, _____(2)=False)
```

Exercise 4. Write the function (level 3)

Write the known-answer cases before the code. Write `share_cases() -> list[tuple[float, float, float | None]]` returning `(part, total, expected)` cases for `share(part, total)`, which returns `part / total`, or `None` when `total` is 0. Your cases must all pass against a correct `share`, and each of three wrong versions must fail at least one of them: one that raises on a zero total, one that returns 0.0 on a zero total, and one that returns a percentage (times 100).

Your code must pass these checks:

- Every case must pass against a correct share. Work the expected values out by hand.
- Add a case with total 0, expecting None. A version that divides by zero must fail it.
- Your zero-total case must expect None, not 0.0, so a version returning 0.0 fails.
- Add a normal case, such as (1, 4, 0.25). A version returning 25.0 must fail it.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Known-answer cases for share(part, total), worked out by hand.
def share_cases() -> list[tuple[float, float, float | None]]: ...
```

Starter code:

```
def share_cases():
    return [(1.0, 2.0, 0.5)]
```

Exercise 5. Architecture design (level 4)

An assistant offers a silver deduplication step keyed on respondent and wave. Plan the known-answer test.

Options: A duplicate in one wave, the same respondent in two waves, a null key, an unchanged respondent, 1,000 rows copied from production.

1. Fixture rows
2. Expected output
3. Proving the test bites

Sources

- pyspark.testing.assertDataFrameEqual signature in 4.0.4 has checkRowOrder=False, rtol=1e-05, atol=1e-08; assertSchemaEqual exists. (verified: inspect.signature on PySpark 4.0.4, September 2026.)

D6-L3 Ask for assumptions, then compare a second approach

Concept

Every piece of code assumes things about its input. Code from an assistant assumes things nobody wrote down. The last two steps of the verification routine bring them into the open.

Ask for the assumptions. After the assistant writes code, ask: "List every assumption this code makes about its input data, one per line." Typical answers for a pipeline step:

- `respondent_id` is unique within a wave.
- `final_weight` is never null and always positive.
- `wave_id` looks like `W` plus two digits.
- Timestamps are in UTC.
- The lookup table has one row per key.

Then **check each one against the real data**, and turn the important ones into checks that run with the pipeline. An assumption you have not checked is a bug waiting for the wave where it stops being true. The list is also a review aid: if an assumption surprises you, the code does something you did not ask for.

The assistant can miss assumptions too. Add your own from the D3 and D4 lessons: nulls in filters, duplicate keys in joins, empty groups, time zones.

Compare a second approach. Compute the same result by an independent route, and compare:

- Spark SQL against the PySpark DataFrame version.
- pandas on a small extract against Spark on the same rows.
- A total from the gold table against a direct aggregate of silver.

"Independent" matters. Asking the same assistant to rewrite the same logic tends to repeat the same mistake. Write the second version yourself, or derive it from a different table.

When the two disagree, do not pick the one you like. Run both on your known-answer data (lesson D6-L2). The one that matches the hand-worked answer is right, and the other has a bug worth understanding.

Why it matters

An assistant's gold aggregation and an analyst's hand-written SQL disagreed by 0.8 points of reach for one region. On the known-answer fixture, the SQL matched the hand calculation. The assistant's version used an inner join to the region lookup, which dropped 212 respondents with a region code missing from the lookup. Asked for its assumptions, the assistant had listed "every region code exists in the lookup". Nobody had checked it.

Worked example: weak code

```
1  # Assistant's code, accepted after it ran
1  gold = silver.join(regions,
     "region_code").groupBy("region").agg(F.sum("final_weight"))
3  gold.write.mode("overwrite").saveAsTable("research_gold.nat_region_totals")
```

1. **Line 2.** An inner join assumes every code is in the lookup. Nobody listed or checked that.

Worked example: strong code

```
1  # Assumption: every region_code in silver exists in regions. Checked:
1  missing = silver.join(regions, "region_code", "left_anti").count()
3  if missing:
4      raise ValueError(f"{missing} respondents have a region code missing
     from the lookup")
5
6  gold = silver.join(regions, "region_code",
     "left").groupBy("region").agg(F.sum("final_weight").alias("w"))
7  # Second approach: the grand total must equal silver's weight total.
8  gold_total = gold.agg(F.sum("w")).first()[0]
9  silver_total = silver.agg(F.sum("final_weight")).first()[0]
2  if abs(gold_total - silver_total) > 1e-6 * silver_total:
11     raise ValueError(f"gold {gold_total} != silver {silver_total}")
```

1. **Line 2.** The assumption, turned into a check that runs every time.
2. **Line 10.** An independent total. A dropped or duplicated row makes them disagree.

Make the assistant state its assumptions, check them on real data, keep the important ones as checks, and confirm results by an independent route.

Recap card

- **Prompt for assumptions** 'List every assumption this code makes about its input data, one per line.' Then check each on real data.
- **Common data assumptions to check** Unique keys per grain, non-null positive weights, id formats, UTC timestamps, one lookup row per key.
- **An independent second approach** SQL vs DataFrame code, pandas vs Spark on an extract, gold totals vs silver aggregates. Not the same assistant rewriting itself.
- **When two methods disagree** Run both on known-answer data. The one matching the hand-worked answer is right.

Practice

Exercise 1. Multiple choice (level 1)

An assistant lists "respondent_id is unique within a wave" as an assumption of its code. What do you do?

- a) Check it on the real data, and add a uniqueness check if it matters
- b) Accept it: the assistant knows the data
- c) Delete duplicates so the assumption holds
- d) Ignore it: assumptions are documentation only

Why?

- r1) An assumption is a claim about data. Only the data can confirm it, and it can stop holding in a later wave.
- r2) Assistants list false assumptions on purpose.

Exercise 2. Predict the output (level 2)

Two methods compute a region's weighted total. What does this print?

```
method_a = 1250.0   # assistant's PySpark, inner join
method_b = 1310.0   # your SQL, left join
known_answer = 1310.0
print("a" if method_a == known_answer else "b")
```

Exercise 3. Fill the gap (level 2)

Check the assumption "every region_code in silver exists in regions".

```
missing = silver.join(regions, "region_code", "_____(1)")._____(2)()
```

Exercise 4. Write the function (level 3)

Write `violated_assumptions(rows: list[dict[str, object]]) -> list[str]` that checks three assumptions and returns the names of those that fail, in this order: `"unique_respondent_per_wave"` (no two rows share `respondent_id` and `wave_id`), `"positive_weight"` (every `final_weight` is a number above 0, not None), `"wave_format"` (every `wave_id` is a string matching `W` followed by exactly two digits). An empty list of rows violates nothing.

Your code must pass these checks:

- The same respondent in two waves is fine. Uniqueness is per wave.
- A duplicate key, a None weight and 'W1' violate all three, reported in order.
- A weight of 0.0 is not positive. Check with > 0, and None with is None.
- No rows, no violations.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
WAVE_PATTERN = ...

# Names of data assumptions that the rows break.
def violated_assumptions(
    rows: list[dict[str, object]],
) -> list[str]: ...
```

Starter code:

```
def violated_assumptions(rows):
    return []
```

Exercise 5. Architecture design (level 4)

Choose the second approach for verifying an assistant's PySpark gold aggregation of weighted reach by region and wave.

Options: Your own Spark SQL query over silver, written without looking at the PySpark code, Ask the same assistant to rewrite it in SQL.

1. Independent method
2. The two results differ for one region
3. After the fix

Sources

- left_anti joins return left rows with no match; used here to check a lookup assumption. (verified: Local run on PySpark 4.0.4 (see D3-L1 claims), September 2026.)

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D6-L1 Read before you run, and check each API

D6-L1 exercise 1

Hints. (1) What can this cell change? (2) The table. (3) Which rule does bronze follow?

Answer. a) The write mode and the target, against the goal and the table's rules Because: r1) Side effects on a production table cannot always be undone. Bronze is append-only.

Side effects first: write mode, target name, DROP, DELETE. Then the rest.

Common wrong answers.

- `b`: Formatting matters later. A destructive write matters now.
- `c`: Speed is secondary to what the cell changes.
- `d`: The result of an overwrite is a replaced table.
- `a|r2`: Right answer, wrong reason. Often the mode is right. You check because the cost of a wrong one is high.

D6-L1 exercise 2

Hints. (1) What do you check the code against? (2) Names before side effects. (3) Run last.

Answer.

1. Restate the goal in one sentence
2. List every function, parameter, table and column
3. Check each name against the docs for your version and the schema
4. Find side effects: write modes, DROP, DELETE
5. Run it on development data in a dev schema

Goal, names, checks, side effects, then a dev run.

Common wrong answers.

- Without a goal stated first, you have nothing to check the code against.
- Find side effects before running, even on dev data.

D6-L1 exercise 3

Hints. (1) A built-in that returns True or False for an attribute. (2) inspect's function for parameters. (3) `hasattr` and `signature`.

Answer.

```python
import inspect
from pyspark.sql import functions as F

hasattr(F, "try_divide")
"validate" in inspect.signature(pd.DataFrame.merge).parameters
```

`hasattr` checks a name. `inspect.signature(...).parameters` checks a parameter.

Common wrong answers.

- g1 = `getattr`: getattr raises if the name is missing. hasattr returns True or False.

D6-L1 exercise 4

Hints. (1) Walk the tree with ast.walk and keep ast.Call nodes. (2) Build the dotted name recursively from Attribute.value down to a Name. (3) Return None when the chain hits anything else.

Answer.

```python
import ast


def _dotted(node: ast.expr) -> str | None:
    """Dotted name for a Name or Attribute chain, else None."""
    if isinstance(node, ast.Name):
        return node.id
    if isinstance(node, ast.Attribute):
        base = _dotted(node.value)
        return None if base is None else f"{base}.{node.attr}"
    return None


def called_names(source: str) -> list[str]:
    """Sorted unique dotted names of every call in the source.

    Args:
        source: Python source code.

    Returns:
        Names such as "F.col" or "spark.read.format".
    """
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.Call):
```

```
            name = _dotted(node.func)
            if name is not None:
                names.add(name)
    return sorted(names)
```

A list of every call is a checklist: each name gets a docs link or a question mark.

D6-L1 exercise 5

Hints. (1) Which line changes existing data? (2) Look at the write mode. (3) Append.

Answer. The bug is on line 5.

Fixed version:

```
from pyspark.sql import functions as F

delivery = spark.read.format("csv").option("header", True).load(path)
delivery = delivery.withColumn("ingestion_timestamp",
    F.current_timestamp())
delivery.write.mode("append").saveAsTable("research_bronze.nat_raw")
```

One word, `overwrite`, turns an append into a replacement of all history.

Common wrong answers.

- Line 4: Adding an ingestion timestamp is part of the bronze standard.
- Line 3: The read is fine. The write replaces history.

D6-L2 Small data with a known answer, failing test first

D6-L2 exercise 1

Hints. (1) What should a test do when the behaviour is missing? (2) Fail. (3) This one did not.

Answer. a) The test does not check the feature Because: r1) A test must fail when the behaviour it checks is missing. Passing without the feature means it asserts something else.

Watch every new test fail once. Then you know it can.

Common wrong answers.

- `b`: The feature does not exist. The test passed anyway, so it is not checking it.
- `c`: A test that cannot fail is not well written.
- `d`: Speed is unrelated.

- `a|r2`: Right answer, wrong reason. Tests do not fail by default. This one should have.

D6-L2 exercise 2

Hints. (1) Where do > and >= differ? (2) Only at equality. (3) Add that case.

Answer. a) A case with a weight exactly equal to the cap Because: r1) > and >= differ only on the boundary, so only a boundary case can tell them apart.

A deliberate break shows which rule your tests miss. Boundaries are the usual gap.

Common wrong answers.

- `b`: Both operators agree far from the cap. Those cases cannot catch this change.
- `c`: Timing has nothing to do with the comparison.
- `d`: They pass with a known bug, so they are incomplete.
- `a|r2`: Right answer, wrong reason. One well-chosen case is enough: the boundary.

D6-L2 exercise 3

Hints. (1) The function you call on the last line. (2) The parameter about row order. (3) `assertDataFrameEqual` and `checkRowOrder`.

Answer.

```
1 from pyspark.testing import assertDataFrameEqual
2
3 expected = spark.createDataFrame([("W01", 0.25)], "wave_id string, share
    double")
4 assertDataFrameEqual(result, expected, checkRowOrder=False)
```

`checkRowOrder=False` is the default. Writing it out documents the intent.

Common wrong answers.

- g2 = `ignoreRowOrder`: The parameter is checkRowOrder. It is False by default in PySpark 4.0.

D6-L2 exercise 4

Hints. (1) A normal case whose share is not 0 or 1, so a percentage differs. (2) A zero total expecting None. (3) Work each expected value out by hand.

Answer.

```
1 def share_cases() -> list[tuple[float, float, float | None]]:
2     """Known-answer cases for share(part, total), worked out by hand.
3
```

```
    Returns:
        (part, total, expected) tuples covering a normal share, a zero
    share and a zero total.
    """
    return [
        (1.0, 4.0, 0.25),
        (0.0, 10.0, 0.0),
        (5.0, 0.0, None),
    ]
```

Good cases are chosen to break wrong code. A case where every version agrees, such as (1, 1, 1.0) for a percentage bug, adds nothing.

D6-L2 exercise 5

Hints. (1) Which rows exercise each rule? (2) Who decides the right answer? (3) How do you prove a test can fail?

Answer.

- Each fixture row exercises one rule of the requirement.
- Expected rows come from you, not from the code under test.
- A deliberate wrong key must fail, or the fixture misses the multi-wave case.

Designed rows, hand-written expectations, and a deliberate break.

Common wrong answers.

- fixture = `sample`: You cannot work out 1,000 rows by hand, and production rarely has every edge case.
- expected = `run`: That makes the test self-confirming: it records the code's output, bugs included.
- prove = `rerun`: A rerun gives the same result. Only a deliberate break shows the test can fail.

D6-L3 Ask for assumptions, then compare a second approach

D6-L3 exercise 1

Hints. (1) Who knows whether it is true? (2) The data. (3) Check it, then keep the check.

Answer. a) Check it on the real data, and add a uniqueness check if it matters Because: r1) An assumption is a claim about data. Only the data can confirm it, and it can stop holding in a later wave.

Check each assumption with a query, then keep the important ones as pipeline checks.

Common wrong answers.

- `b`: The assistant has never seen your data. It guessed.
- `c`: Silently deleting rows hides a delivery problem. Check first, then decide.
- `d`: An unchecked assumption is a future bug.
- `a|r2`: Right answer, wrong reason. The assumptions are usually sensible guesses. You check because guesses can be wrong.

D6-L3 exercise 2

Hints. (1) Which method equals the known answer? (2) 1310. (3) Method b.

Answer.

```
b
```

The known answer settles disagreements. Method a lost rows, most likely in its inner join.

Common wrong answers.

- `a`: Method a gives 1250, which does not match the hand-worked 1310.

D6-L3 exercise 3

Hints. (1) Which join keeps left rows without a match? (2) Then count them. (3) `left_anti` and `count`.

Answer.

```
missing = silver.join(regions, "region_code", "left_anti").count()
```

A left anti join returns silver rows with no match in regions. Zero means the assumption holds.

Common wrong answers.

- g1 = `inner`: An inner join keeps the matches. You want the rows with no match.

D6-L3 exercise 4

Hints. (1) Uniqueness uses the pair (respondent_id, wave_id). (2) Positive means > 0 and not None. (3) Use re.fullmatch so 'W145' does not pass.

Answer.

```
import re

```

```python
WAVE_PATTERN = re.compile(r"W\d{2}")


def violated_assumptions(rows: list[dict[str, object]]) -> list[str]:
    """Names of data assumptions that the rows break.

    Args:
        rows: Records with respondent_id, wave_id and final_weight.

    Returns:
        Violated assumption names in a fixed order.
    """
    violated: list[str] = []
    keys = [(r["respondent_id"], r["wave_id"]) for r in rows]
    if len(keys) != len(set(keys)):
        violated.append("unique_respondent_per_wave")
    weights = [r["final_weight"] for r in rows]
    if any(w is None or not isinstance(w, (int, float)) or w <= 0 for w in weights):
        violated.append("positive_weight")
    waves = [r["wave_id"] for r in rows]
    if any(not isinstance(w, str) or WAVE_PATTERN.fullmatch(w) is None for w in waves):
        violated.append("wave_format")
    return violated
```

Each assumption becomes one small check with a name. The names go into the run log, so a failure says which assumption broke.

D6-L3 exercise 5

Hints. (1) What makes a second method independent? (2) What decides who is right? (3) What catches the next regression?

Answer.

- Independent code avoids repeating the same reasoning, and the same mistake.
- Hand-worked answers decide disagreements.
- A cheap reconciliation total stays in the pipeline to catch the next regression.

Independent method, known answer as referee, and a lasting reconciliation check.

Common wrong answers.

- route = `rewrite`: The same assistant tends to repeat its own assumptions, such as the inner join.
- disagree = `average`: An average of a right and a wrong number is wrong.

- keep = `drop_sql`: The next change can break it again. Keep a cheap independent check.